Capstone Deployment Architecture

Iguana64 / Falcon64 Ensemble

https://secretbloodtest.com (AWS Deployment)

---

1. System Overview

SecretBloodTest.com is a cloud-hosted deep learning inference system designed to classify many types of blood cells using an ensemble of custom convolutional architectures: Falcon64 and Iguana64 variants.

The deployed system accepts user-uploaded microscope images, performs preprocessing and ensemble inference, and returns predicted cell type probabilities in near real time.

The system is production-deployed on AWS using Elastic Beanstalk, an Application Load Balancer, EC2 Auto Scaling, and containerized Python inference services.

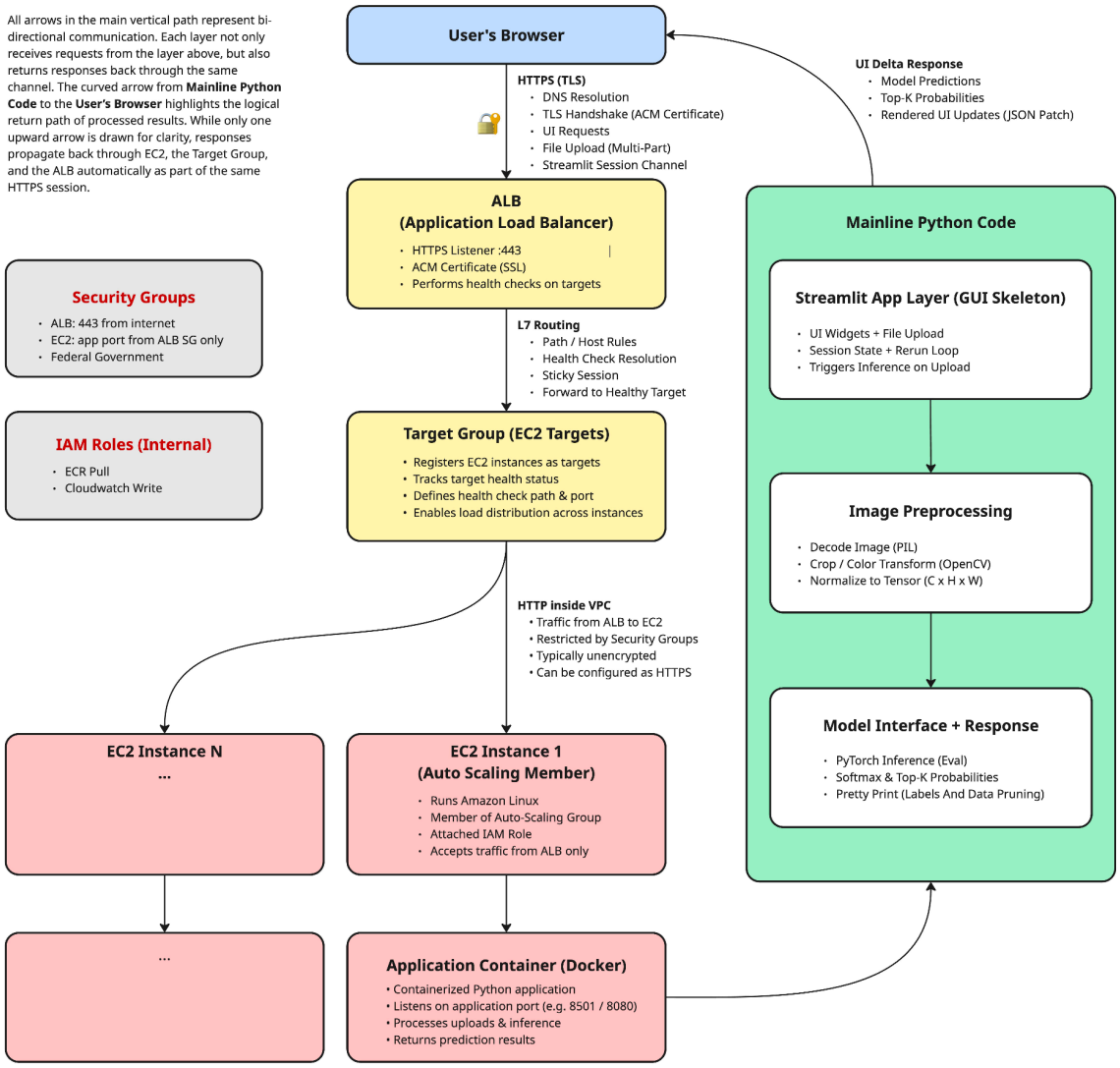
The application currently serves as a demonstration-grade prototype, not a clinically certified diagnostic tool.

---

2. Major Components and Data Flow

The system contains the following major runtime components:

• User Browser (Image Upload + UI Session)
• Application Load Balancer (TLS Termination + Routing)
• Target Group (Health Tracking + Load Distribution)
• EC2 Instances (Auto Scaling Members)
• Application Container (Docker)
• Mainline Python Code (Streamlit + preprocessing + PyTorch ensemble)



---

3. Model Architecture and Evolution

The final deployed system uses a six-member ensemble:

- Falcon Gold
- Falcon Silver
- Falcon Bronze
- Iguana Gold
- Iguana Silver
- Iguana Bronze

Each model is built from depthwise-separable residual bottleneck blocks with Group Normalization, Squeeze-Excitation recalibration, and Generalized Mean Pooling. The Iguana architecture includes a dual-embedding fusion branch with gated feature fusion.

The model suite evolved across many phases:

• Early Toad64 prototypes
• RoadRunner experimental variants
• Falcon64 structured separable CNN
• Iguana64 dual-branch fusion models
• Extensive experiments with auxiliary knowledge branches

The final ensemble achieves approximately **93% accuracy** on the held-out test split of the primary training dataset.

For comparison, a simple “cookie-cutter” baseline CNN achieves only ~64% accuracy under identical conditions.

---

4. Honest Performance Assessment

While ensemble accuracy is strong on in-domain testing data, performance is highly dependent on:

• Microscope type and calibration
• Staining profile
• Brightness and contrast levels
• Background illumination characteristics

The model performs well on images derived from the dataset distribution it was trained on. However, even modest shifts in brightness or contrast degrade performance significantly.

The model is also limited in morphological robustness. It does not explicitly understand:

• Nucleus-to-cytoplasm ratio
• Nuclear shape irregularity
• Cytoplasmic granule density
• Boundary structure
• Morphological abnormalities

The current system is primarily texture-driven rather than morphology-aware.

This limits generalization to external laboratory environments.

5. Proposed Morphology-Infused Segmentation Network

To address the above limitations, a proposed future project introduces a pixel-wise classification neural network.

For each pixel in an image:
- Extract a fixed-size local window (e.g., 63×63)
- Classify the center pixel as:

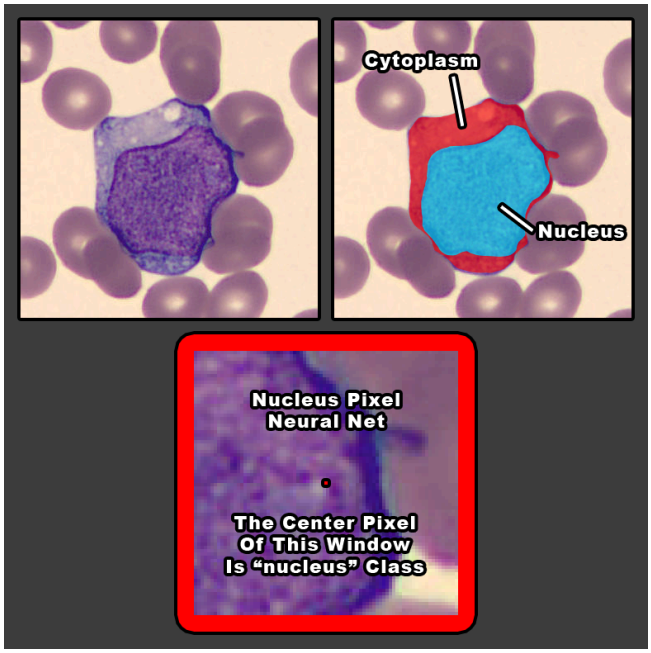
1. Nucleus
2. Cytoplasm
3. Irrelevant

This produces a segmentation mask that separates nucleus and cytoplasm regions.

From this segmentation, explicit morphology features can be computed.

These features could then be infused into a gated auxiliary embedding branch, extending the Iguana64 fusion mechanism.

This shifts the system from correlation-driven classification toward morphology-aware biological reasoning.



6. Data Storage and Artifact Management

Current deployment stores:

• Docker container images in ECR
• Model state_dict artifacts embedded in image
• Logs and metrics in CloudWatch

Uploaded images are processed in memory and not persisted by default. For retraining purposes, curated datasets can be stored in S3 with version control.

All model artifacts are versioned via Docker image tags and can be rolled back safely.

---

7. Model Lifecycle and Retraining

Current deployment is inference-only.

Retraining is condition-driven rather than fixed-interval.

Triggers for retraining include:

• Degraded performance on new datasets
• Distribution shift (new microscope formats)
• Addition of new classes
• Significant domain expansion

Retraining requires labeled images from diverse microscope sources.

Evaluation includes:

• Held-out validation set
• Independent test split
• Accuracy metrics
• Ensemble agreement analysis

Only models exceeding existing performance without regression are promoted.

Deployment of retrained models occurs via:

1. Zip Project Folder
2. Push to Elastic Beanstalk

Note, you cannot zip the folder, cd into the project root and execute:

```
zip -r ../my_zip.zip . \
  -x "*.git*" \
  -x "*.venv*" \
  -x "__pycache__/*" \
  -x "*.DS_Store" \
  -x "*.pyc"
```

---

8. Scalability, Resilience, and Stress Handling

The architecture is designed for:

Scalability:
• Auto Scaling Group adds EC2 instances under load
• Horizontal scaling for traffic spikes

Low Latency:
• Models preloaded into memory
• Inference performed entirely in RAM
• Internal VPC routing

Fault Tolerance:
• ALB health checks remove unhealthy instances
• Auto Scaling replaces failed nodes
• Rolling deployments prevent downtime

Stress Cases Considered:
• High concurrent upload bursts
• EC2 instance failure
• Container crash recovery
• Heavy inference queue buildup

Architecture can scale to GPU-backed EC2 instances if needed.

9. Alternative Architectures Considered

• GPU-backed EC2 for faster inference
• ECS/Fargate container orchestration
• Lambda-based burst inference
• Hybrid segmentation + classification system

The chosen Elastic Beanstalk architecture balances simplicity, cost-efficiency, and deployment control.

I spent significant time trying to deploy to App Runner, which is not suitable for Streamlit.

---

10. Cost Estimate

Estimated monthly cost (moderate usage):

• 1–2 t3.medium EC2 instances
• Application Load Balancer
• ECR storage
• CloudWatch logging

Approximate range: $70–$180 USD/month depending on scaling.

Development effort reflects extensive experimentation across multiple architectural generations.

Currently I am using t3.small, I don’t forecast the need to switch to medium yet.

---

11. Conclusion

The Iguana64 / Falcon64 ensemble represents a robust in-domain prototype achieving strong dataset-specific performance (~93% accuracy). However, it remains sensitive to domain shifts and lacks explicit morphological reasoning.

Future work focuses on morphology-infused segmentation via a pixel-level classifier, feature extraction, and structured biological embedding fusion. Improved billing and payment.

The current AWS deployment is scalable, resilient, and production-capable for demonstration use, with clear pathways for improvement toward greater robustness and generalization.

12. Streamlit Limitations

Ultimately, the dam started to form cracks in terms of Streamlit maintaining an accurately manage-able interface. So, the plan is to also consider flask.